「定量的マクロ経済学と数値計算」勉強会第三章: 動的計画法

鈴木瑞洋

2026/04/05

ベルマン方程式を使って動学モデルを解く方法（動的計画法）を説明する

- ロビンソン・クルーソー経済の導入
- ベルマン方程式
- 有限期間モデル
  - 後ろ向き帰納法
  - スプライン補間を使った関数の近似
- 無限期間モデル
- 無限期間モデルを解くための価値関数反復法 (VFI)
- VFIより早い政策関数反復法 (PFI)
- リスクの導入（それに伴う状態変数の追加）
- 構造推定

ロビンソン・クルーソー経済

- 最適な消費・貯蓄計画をたてる

$$\max_{\{c_t, k_{t+1}\}_{t=0}^{T}} \quad \sum_{t=0}^{T} \beta^t u(c_t)$$

$$\text{subject to} \quad c_t + k_{t+1} \leq f(k_t) \quad (t = 0, 1, \ldots, T)$$

  - 最後の期（$t = T$）では貯蓄をしても意味がないので、$k_{T+1} = 0$
  - 借り入れはできない：$k_t \geq 0 \quad (t = 1, \ldots, T)$
  - 資源は無駄にしないので、制約は等号で成立する：$c_t + k_{t+1} = f(k_t)$

ベルマン方程式

- 価値関数 $V_t(k_t)$: t期においてk_tだけ資本を保有していた場合にt期以降で達成できる最大効用

$$V_t(k_t) = \max\{u(c_t) + \beta u(c_{t+1}) + \cdots + \beta^{T-t} u(c_T)\}$$

これはk_tという状態変数のもとで最適な消費をt期からT期まで求める問題

- これは次のように書き換えられる

$$\begin{aligned} V_t(k_t) &= \max\{u(c_t) + \beta u(c_{t+1}) + \cdots + \beta^{T-t} u(c_T)\} \\ &= \max\{u(c_t) + \beta \max\{u(c_{t+1}) + \cdots + \beta^{T-t-1} u(c_T)\}\} \\ &= \max\{u(f(k_t) - k_{t+1}) + \beta V_{t+1}(k_{t+1})\} \end{aligned}$$

これは「$t+1$期からは$k_{t+1}(= f(k_t) - c_t)$という状態変数からスタートして最適な消費を$t+1$期からT期まで決める (*)」として、今期の状態変数k_tのもとでの最適な消費c_tを決める、という問題
$\Rightarrow$ ざっくりいうと、(*) が成り立つもとで、「t期にどれだけ消費・貯蓄をするか」という問題に単純化されている

有限期間モデルでの後ろ向き帰納法

- 前のページからちょっと書き換えて

$$V_t(k_t) = \max_{0 \le c_t \le f(k_t)} \{u(c_t) + \beta V_{t+1}(f(k_t) - c_t)\}$$

なので、$V_{t+1}(k_{t+1})$がわかれば、最適なc_tを求められる

- 最後の期$t = T$では、

$$\begin{aligned} V_T(k_T) &= \max_{0 \le c_T \le f(k_T)} u(c_T) \quad (\text{次の期がないので}V_{T+1}\text{の項はない}) \\ &= u(f(k_T)) \quad (\text{次の期のために貯蓄をしないので}k_{T+1} = 0) \end{aligned}$$

- $t = T - 1$期では

$$V_{T-1}(k_{T-1}) = \max_{0 \le c_{T-1} \le f(k_{T-1})} \{u(c_{T-1}) + \beta V_T(f(k_{T-1}) - c_{T-1})\}$$

を解く

3次のスプライン補間

- 各期の状態変数k_tを離散化すれば（$k_{t,i} \in \{k_1, \ldots, k_N\}$）、$V_{T-1}(k_{T-1,i})$は解ける（$V_T(k_T)$が解析的にわかっているので）
- ただし、このままではグリッド上にないk_{T-1}での$V_{T-1}(k_{T-1})$はわからない
- $t = T - 2$期では

$$V_{T-2}(k_{T-2}) = \max_{0 \le c_{T-2} \le f(k_{T-2})} \{u(c_{T-2}) + \beta V_{T-1}(f(k_{T-2}) - c_{T-2})\}$$

  を解くが、$V_{T-1}(k_{T-1})$がすべてのk_{T-1}でわからないと解けない
- 内挿法で$V_{T-1}(k_{T-1})$を近似する！（教科書2.8.1も参照）
- 3次のスプライン補間：
  - $k_T \in [k_i, k_{i+1}]$について、$V_{T-1}(k_{T-1})$を$\tilde{V}_{T-1}(k_{T-1}) = a_i + b_i(k_{T-1} - k_i) + c_i(k_{T-1} - k_i)^2 + d_i(k_{T-1} - k_i)^3$で近似
  - a_i, b_i, c_i, d_iは各グリッド上で$\tilde{V}_{T-1}(k_{T-1})$が連続でこの一階微分、二階微分も連続になるように決められる
  - （ざっくりいうと、$\tilde{V}_{T-1}(k_{T-1})$がいい感じに滑らかになるように決められる）

無限期間モデル

- 無限期間モデルに拡張：

$$\max_{\{c_t, k_{t+1}\}_{t=0}^{\infty}} \quad \sum_{t=0}^{\infty} \beta^t u(c_t)$$

$$\text{subject to} \quad c_t + k_{t+1} = f(k_t) + (1-\delta)k_t, \quad k_0 \text{ given}$$

- ベルマン方程式に書き直すと

$$V(k) = \max_{k'} \{u(f(k) + (1-\delta)k - k') + \beta V(k')\}$$

  - 添字tがなくなっているのは、無限期間モデルでは残存期間は常に無限なので、今が何期目かは意思決定に必要でないため
- 一階条件は
  - ベルマン方程式の右辺の最適化問題を考えて

$$u'(f(k) + (1-\delta)k - k') = \beta V'(k')$$

  - 包絡線定理より、ベルマン方程式の両辺をkで微分して

$$V'(k) = (1 - \delta + f'(k))u'(f(k) + (1-\delta)k - k')$$

価値関数反復法（VFI）

- 適当に各k_iでの$V^{(0)}(k_i)$を決めて、以下のように$V^{(n)}(k)$を更新する

$$V^{(n+1)}(k_i) = \max_{k'}\{u(f(k_i) + (1-\delta)k_i - k') + \beta V^{(n)}(k')\}$$

- 各k_iでの$V^{(n+1)}(k_i)$と$V^{(n)}(k_i)$が十分小さくなったらおしまい
- 各k_iでの最適なk'を求める方法：
  - グリッドサーチ：k'も離散化して、右辺を最大化するk'を求める
  - パラメトリック・アプローチ：k'を連続として扱い、$V^{(n)}$を内挿法で近似しながら右辺を最大化するk'を求める

政策関数反復法（PFI）

- VFIより早い、政策関数の繰り返し計算を使うアルゴリズム
- いい感じに各k_iでの政策関数$g^{[0]}(k_i)$を決めて、
  1. 適当に各k_iでの$\tilde{V}^{(0)}(k_i)$を決めたあとに
     $$\tilde{V}^{(n)}(k) = u(f(k) + (1-\delta)k - g^{[0]}(k)) + \beta \tilde{V}^{(n-1)}(g^{[0]}(k))$$
     を収束するまで繰り返して$\tilde{V}(k)$を得る
  2. 各k_iで
     $$V^{[0]}(k_i) = \max_{k'} \{u(f(k_i) + (1-\delta)k_i - k') + \beta \tilde{V}(k')\}$$
     の解となるk'を求め、それを新しい政策関数 $k' = g^{[1]}(k_i)$とする
  3. ステップ1と2を、価値関数$V^{[m]}$ (または政策関数$g^{[m]}$)が収束するまで繰り返す
- （ここで上添字が[]のときはiterationが何回目かを表していて、()のときは各iteration内の価値関数の反復計算が何回目かを表している）
- ステップ1では最適化問題を解いてなくて、ただ繰り返し計算しているだけなので早い
- 直感的には、「いい感じ」の価値関数をぱぱっと計算した後で重たい最適化を解くことで iteration の数を減らせる、ということ？

VFI vs PFI と収束条件

- ここまでに出てきた計算アルゴリズム、2つの次元がある？
  1. VFI vs PFI
  2. 収束条件として価値関数を使うか政策関数を使うか

Figure 1: 計算時間の比較（カッコ内はiterationの回数）

アルゴリズム \ 収束条件	価値関数	政策関数
VFI	49.105 ms （205回）	3.042 ms （13回）
PFI	9.870 ms （2回）	9.853 ms （2回）

- VFIを使うとき、政策関数を収束条件に使ったほうが収束が早い（ただし価値関数の誤差は大きくなる）
- PFIを使うとき、たしかにVFIよりiterationの回数は減るが、計算時間は増える（各iteration内で価値関数の繰り返し計算もしないといけないため？）
  - 実際、価値関数の繰り返し計算の収束条件を緩くすることで、最終的な誤差を増やすことなく計算時間を1.2msくらいまで減らせた

構造推定

- 「経済主体の意思決定に基づく理論モデルを構築したうえでパラメータを推定して分析する方法を構造推定と呼ぶ」
- 教科書にある例をめっちゃ抽象化するとこんな感じ：
  1. 経済モデルを立てる（ここでは不確実性のある動学モデル）
  2. いくつかのパラメータをカリブレーションから求める（違うデータから求めたり、先行研究からとってきたり、直感的な値をとってきたり）
  3. 残りのパラメータを推定する
     - 3.1 あるパラメータのもとでモデルを解き政策関数を求める（ここで、この章で学んだことを使う）
     - 3.2 政策関数を使ってシミュレートされたデータと実際のデータとの乖離を測る
     - 3.3 この乖離が小さくなるようなパラメータを求める
- 私がいつもわからないのは
  - どのパラメータはカリブレーションで求めて良くてどのパラメータを推定すべき？
  - パラメータの識別条件は何？(どんなときに推定にバイアスが生まれる？)